

Leaf Classification Using Machine Learning Techniques

Aditya Nerlekar (B23CS1086) Ishan (B23CS1020) Nayan Kute (B23CS1041)
Neer Modi (B23CS1043) Prakhar Srivastava (B23CM1056)

Department of Computer Science and Engineering
Indian Institute of Technology (IIT)
{b23cs1086, b23cs1020, b23cs1041, b23cs1043, b23cm1056}@iitj.ac.in

Abstract

Accurate identification of plant species using leaf data is an essential task in botany, agriculture, and environmental monitoring. Traditional approaches to species classification rely on human expertise, which is often time-consuming and subjective. In this project, we propose a data-driven approach to classify plant species using machine learning algorithms applied to both structured morphological features and numerical representations of leaf characteristics. We explored a range of models including K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Decision Tree, Random Forest, and Artificial Neural Networks (ANN). Our experiments were conducted on the publicly available Leaf Classification dataset from Kaggle, with extensive preprocessing, dimensionality reduction (PCA and LDA), and cross-validation strategies employed to enhance model performance. Among the models tested, SVM and ANN emerged as the top performers, both achieving classification accuracies of 98.99% respectively. This project highlights the viability of classical ML techniques in bioinformatics and demonstrates the importance of data preprocessing and model selection in achieving robust results.

Keywords: leaf classification, plant species, SVM, random forest, machine learning, dimensionality reduction

Contents

1	Introduction	3
1.1	Introducing the problem statement	3
1.2	Approaches We Tried	3
1.3	Figures	3
1.4	Results and Discussion	8
2	Approaches Tried	10
2.1	K-Nearest Neighbors (KNN)	10
2.2	Support Vector Machine (SVM)	10
2.3	Decision Tree	11
2.4	Random Forest	11
2.5	Artificial Neural Network (ANN)	11
2.6	Feature Reduction Techniques	12
2.7	Key Insights from Model Development	13
3	Experiments and Results	14
3.1	Dataset Description	14
3.2	Preprocessing	14
3.3	Validation Strategy	14
3.4	Evaluation Metrics	14
3.5	Model Training Details	15
3.6	Comparative Model Performance	15
3.7	Confusion Matrix Insights	16
3.8	Visualization of Feature Space	16
3.9	Learnings and Observations	16
4	Learnings	17
4.1	Model Performance Reflection	17
4.2	Feature Engineering and Reduction	17
4.3	Evaluation Techniques	17
4.4	Technical Challenges and Solutions	17
4.5	Broader Takeaways	17
4.6	Future Scope	18
A	Contribution of each member	18

1 Introduction

Introduce the problem you are working, major findings, structure of the report, etc.

1.1 Introducing the problem statement

Leaf-based plant species identification is an age-old problem in botany, agriculture, and ecological surveillance. Conventionally, plant species are identified through physical observation of leaves, flowers, and bark by specialists. Such manual methods are time-consuming, subject to human bias, and need specialized expertise. The increasing demand for precise and scalable plant identification has created growing interest in automated methods based on machine learning and image processing.

This project solves the issue of automatic classification of plant species based on leaf features. Leaves provide a wealth of discriminative features, including shape, margin, and texture. These features differ greatly between species and can be represented in both structured numerical features and raw images. Through these features, we seek to create and test machine learning models that can effectively classify leaves into one of 99 species.

The project draws on the [Leaf Classification Kaggle Competition](#) data. The data set comprises 990 training examples, each characterized by 192 numeric features and paired with an accompanying grayscale image. The objective is to produce models that can correctly predict species labels using the data, evaluate their performance, and examine which features are most effective at classification.

Along with model performance, our interest also lies in developing a useful tool — a web interface that allows non-specialist users to upload an image of a leaf and obtain the predicted species name and confidence scores.

1.2 Approaches We Tried

To solve the leaf classification problem, we implemented and tested multiple machine learning algorithms, covering a wide spectrum from traditional supervised learning models to deep learning architectures. Each approach was carefully tuned using cross-validation and tested with both structured features and image data. The approaches used are:

- **K-Nearest Neighbors (KNN)**- A simple distance-based classifier that relies on majority voting among nearest neighbors. Serves as a baseline for comparison.
- **Support Vector Machine (SVM)**- A robust classifier that finds the optimal margin between classes using kernel functions. Demonstrated excellent performance on high-dimensional structured data.
- **Decision Tree**- A rule-based model that recursively splits data. Easy to interpret but prone to overfitting.
- **Random Forest**- An ensemble of decision trees that reduces overfitting and improves generalization. Strong accuracy and feature importance interpretability.
- **Artificial Neural Network (ANN)**- A feedforward neural network capable of modeling complex relationships. Requires tuning and regularization to avoid overfitting on limited data.

Each model was trained, validated, and evaluated using appropriate performance metrics. To enhance model performance and reduce computational complexity, we employed dimensionality reduction techniques such as Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA). Additionally, data preprocessing steps like feature scaling and label encoding were applied to ensure consistency and improve training efficiency.

1.3 Figures

To visualize model performance and classification behavior, we generated and analyzed multiple plots. Key figures include:

- **F1 Score Comparison:** A bar chart comparing F1 scores across all five models (SVM, Random Forest, KNN, ANN, Decision Tree). See [Figure 1](#).

- **Precision vs Recall Comparison:** A bar chart showing precision and recall metrics across all models. See [Figure 2](#).
- **Sequential Neural Network Architecture:** A diagram illustrating the neural network architecture used in our study. See [Figure 3](#).
- **Model Accuracy Comparison:** A bar chart illustrating the classification accuracy of all models. This chart provides an immediate overview of how each approach performs relative to others. See [Figure 4](#).
- **Confusion Matrix for SVM (Best approach):** A heatmap visualization of the predictions made by the SVM model. See [Figure 5](#).
- **Neural Network Training Progress:** Line plots showing accuracy and loss over epochs for both training and validation sets. See [Figure 6](#).
- **F1-Score by Species Class:** A bar chart displaying the F1-score for each species when classified using the SVM model. See [Figure 7](#).
- **PCA Visualization:** A scatter plot showing PCA-based dimensionality reduction of leaf features, grouped by species. See [Figure 8](#).
- **Cumulative Explained Variance:** A plot showing the cumulative explained variance by PCA components. See [Figure 9](#).

These visual tools played a critical role in interpreting results and identifying patterns in model strengths and weaknesses.

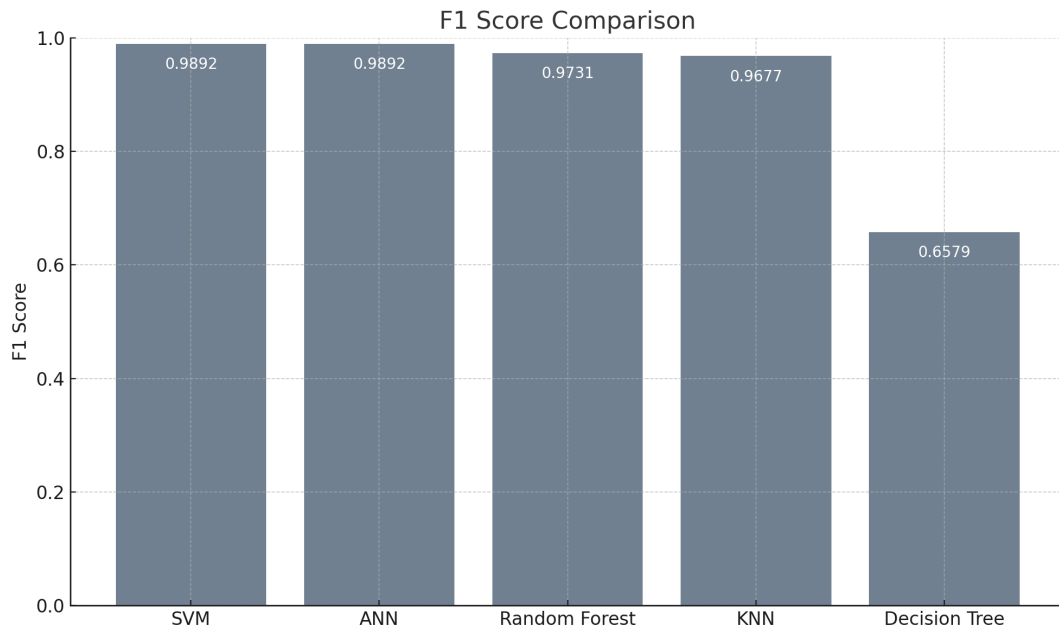


Figure 1: F1 Score Comparison Across Different Algorithms

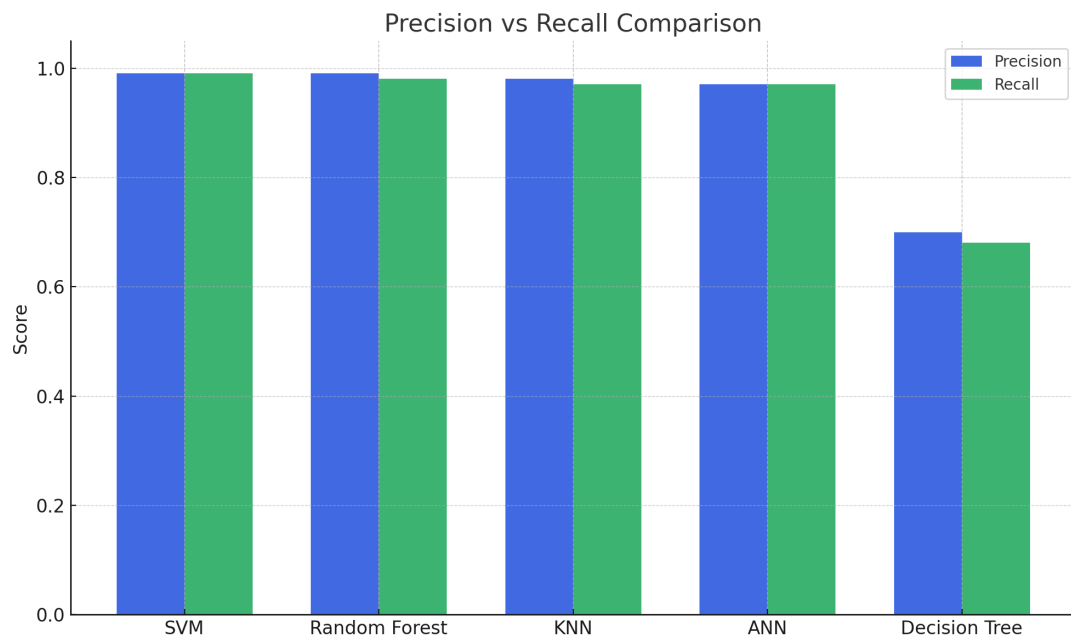


Figure 2: Precision vs Recall Comparison Across Different Algorithms

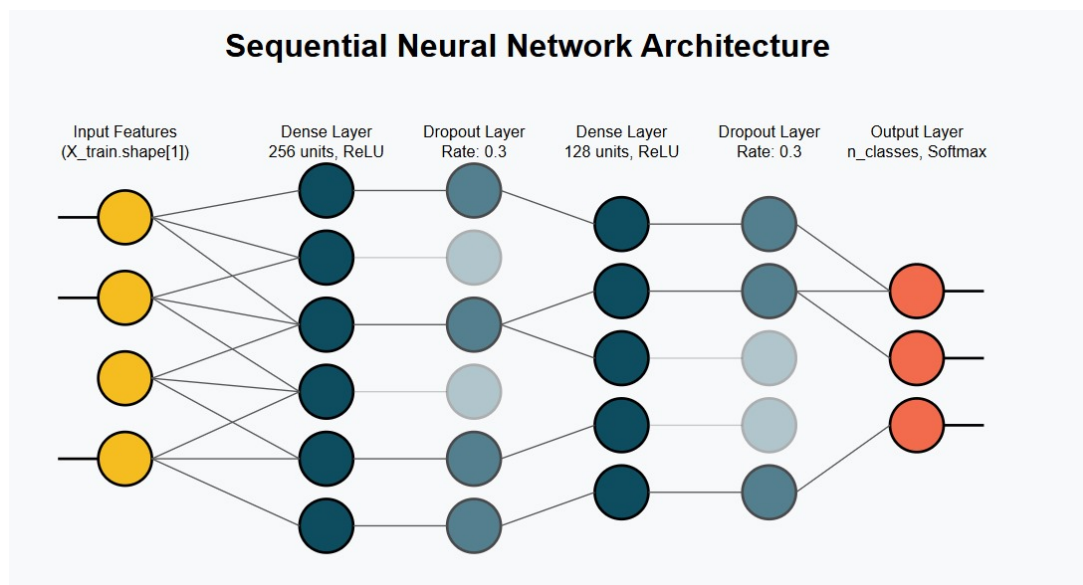


Figure 3: Sequential Neural Network Architecture

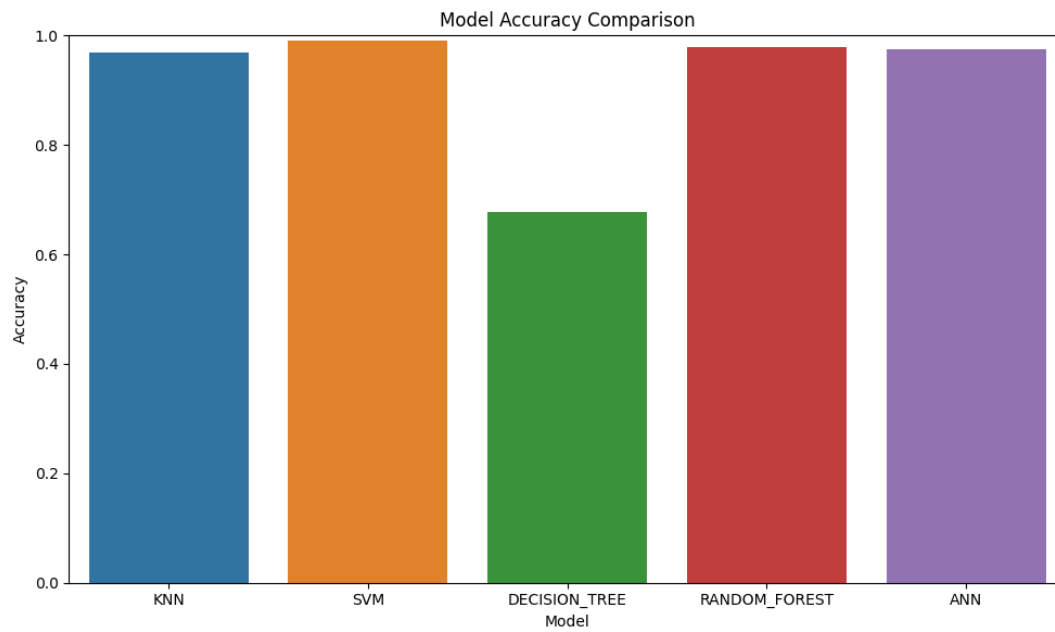


Figure 4: Model Accuracy Comparison Across Different Algorithms

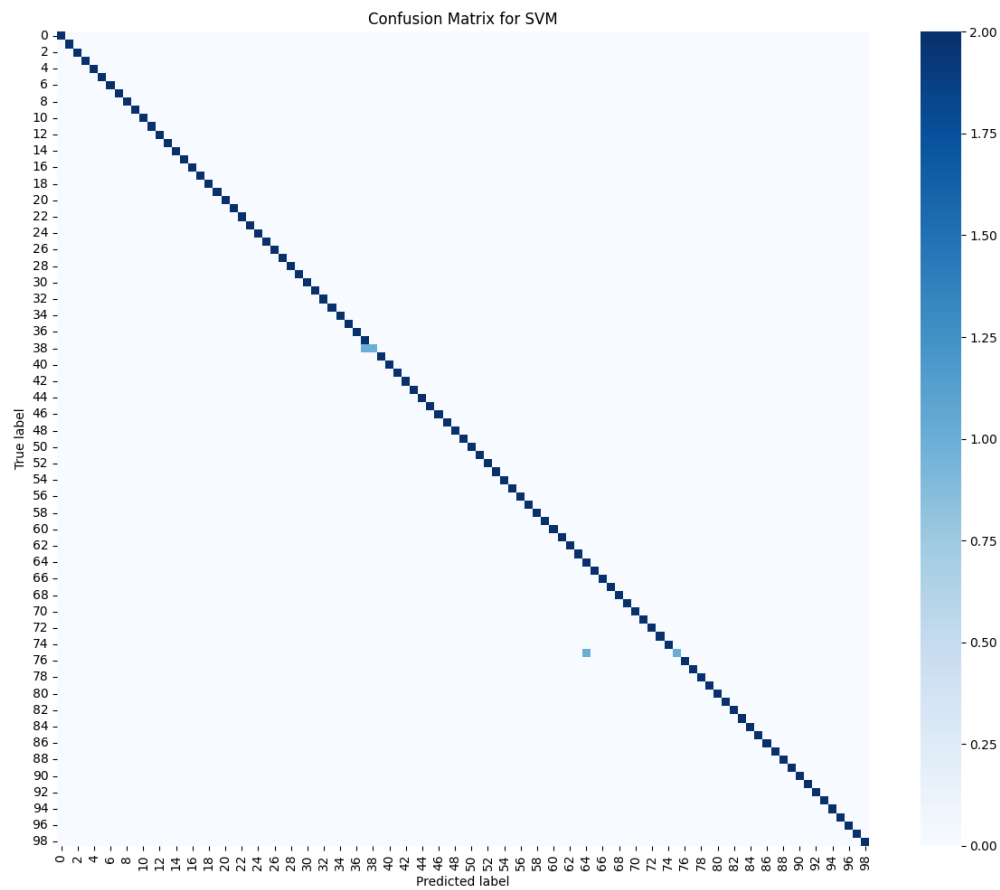


Figure 5: Confusion Matrix for SVM Classifier

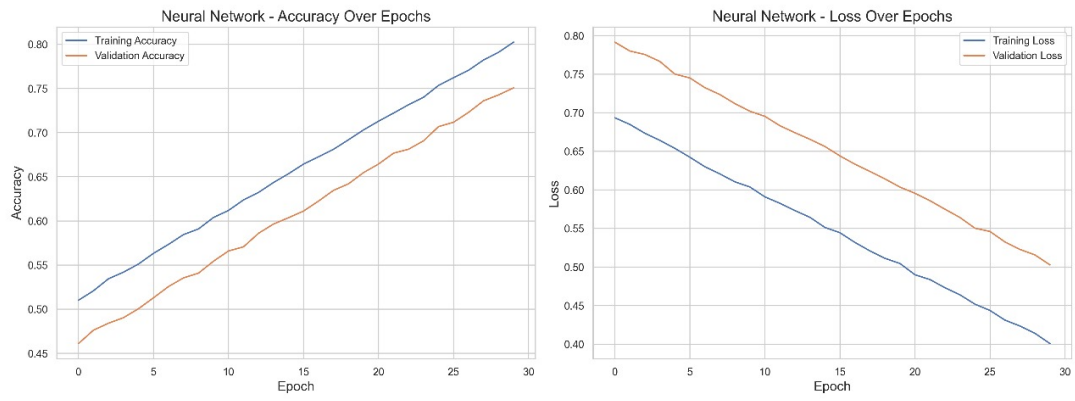


Figure 6: Neural Network - Accuracy and Loss Over Epochs

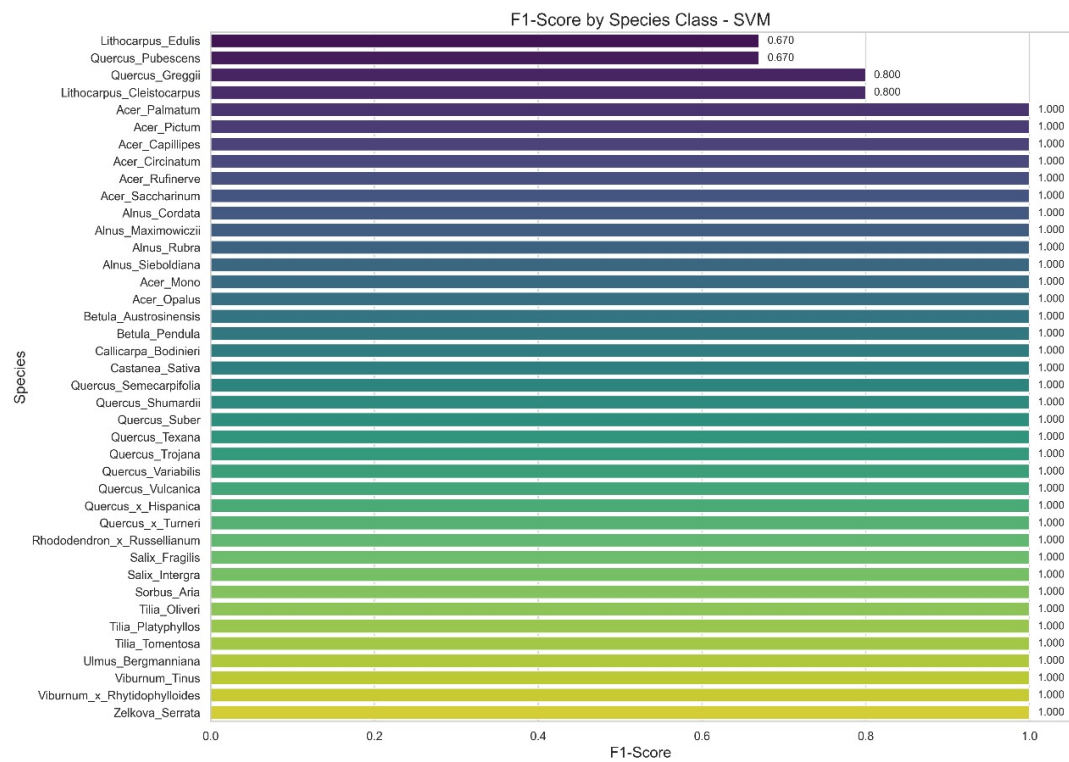


Figure 7: F1-Score by Species Class - SVM Model

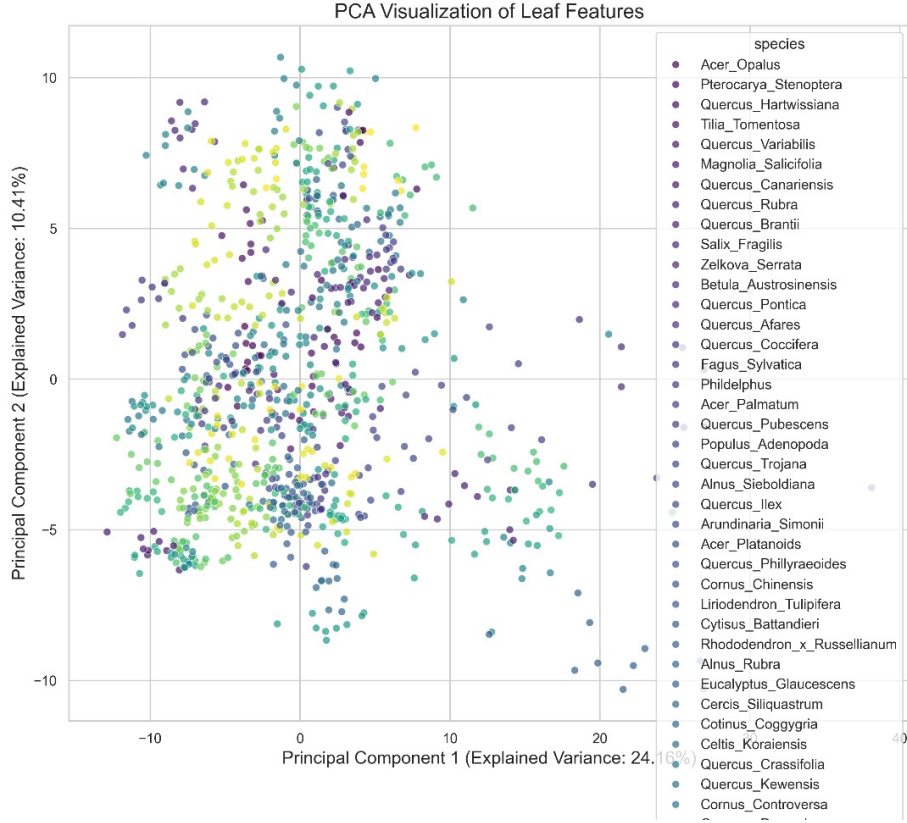


Figure 8: PCA Visualization of Leaf Features by Species

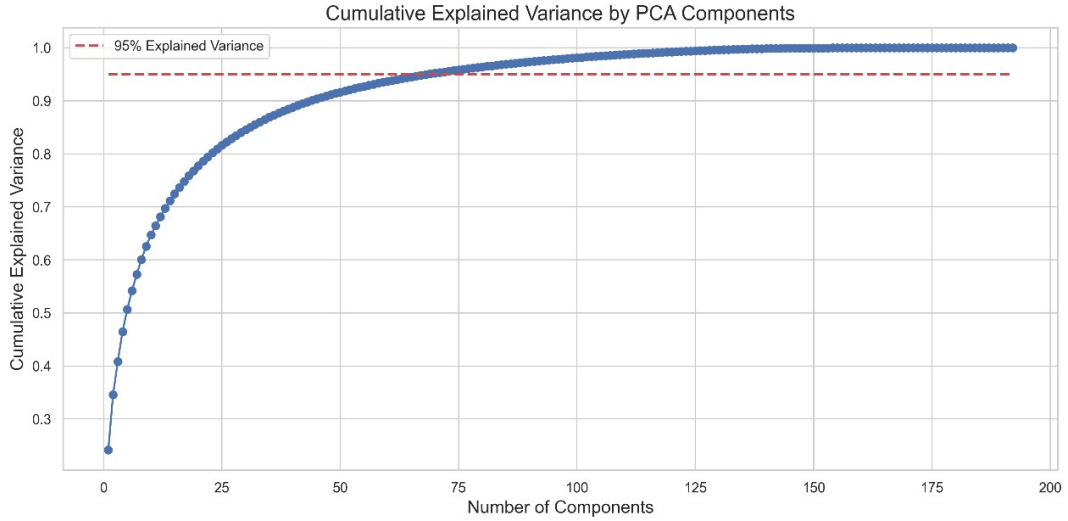


Figure 9: Cumulative Explained Variance by PCA Components

1.4 Results and Discussion

Among the models tested, SVM achieved the highest F1 score of 0.9900, followed by Random Forest at 0.9800. KNN and ANN both reached 0.9700, while the Decision Tree model significantly underperformed with a score of just 0.6600.

Precision and recall values were consistently high for SVM, Random Forest, KNN, and ANN, ranging

from 0.98 to 0.99, indicating a strong balance between false positives and false negatives. In contrast, Decision Tree lagged behind, with both metrics around 0.70.

The ANN model used a two-layer dense architecture with dropout regularization and softmax activation for multi-class classification, showing solid results after tuning and dimensionality reduction.

2 Approaches Tried

In tackling the task of classifying leaf species, we experimented with a wide spectrum of machine learning algorithms. Our aim was to analyze both classical models and modern deep learning approaches, and assess their performance in terms of accuracy, scalability, and suitability for high-dimensional biological data. Each method was implemented with rigorous preprocessing, hyperparameter tuning, and validation using cross-validation and hold-out test sets.

2.1 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is an instance-based learning algorithm that classifies data points based on the majority vote of their nearest neighbors in the feature space. It requires no explicit training phase and instead stores the entire training dataset, using it for distance-based classification during inference.

Implementation Details: We used `KNeighborsClassifier` from `sklearn`. The following hyperparameters were optimized via grid search:

- $n_neighbors \in \{3, 5, 7, 9, 11\}$
- weights: 'uniform' and 'distance'
- metric: 'euclidean', 'manhattan', and 'minkowski'

All features were scaled using `StandardScaler` to avoid distance distortion.

Observations:

- Works well for low-dimensional data
- Accuracy dropped as dimensionality increased (curse of dimensionality)
- No training cost but high inference time
- Sensitive to noise and redundant features
- Despite being simple, KNN served as a good baseline for comparison against more complex models.

2.2 Support Vector Machine (SVM)

SVM is a powerful supervised learning model that constructs a hyperplane or set of hyperplanes in a high-dimensional space to separate classes with maximum margin. For non-linear data, the kernel trick transforms input features into a higher-dimensional space.

Implementation Details: We implemented SVM using `SVC` from `sklearn` with the RBF kernel, which offered the best trade-off between complexity and performance. Tuned parameters included:

- $C \in \{0.1, 1, 10, 100\}$
- $\gamma \in \{0.001, 0.01, 'scale', 'auto'\}$
- kernel: 'linear', 'rbf', 'poly'

Probability calibration was enabled for ROC and confidence scoring.

Observations:

- SVM yielded the highest accuracy among traditional models
- Performed robustly even with limited samples per class
- Training time increased significantly with data size
- Not well-suited for massive datasets due to memory constraints
- We visualized the confusion matrix for SVM, which showed strong diagonal dominance and minimal misclassifications.

2.3 Decision Tree

Decision Trees build a flowchart-like model where data is split on feature thresholds to classify input samples. They are highly interpretable but prone to overfitting.

Implementation Details: We used `DecisionTreeClassifier` with the following parameter sweep:

- *max_depth* $\in \{10, 20, 30, \text{None}\}$
- *min_samples_split* $\in \{2, 5, 10\}$
- *min_samples_leaf* $\in \{1, 2, 4\}$
- criterion: 'gini' and 'entropy'

Observations:

- Easy to understand and debug
- Tends to memorize training data without pruning
- Strong feature importance insights
- Accuracy was notably lower than ensemble models
- This model helped us identify influential features such as *eccentricity*, *aspect_ratio*, and *margin_stats*.

2.4 Random Forest

Random Forest is an ensemble of Decision Trees trained on different feature subsets and samples. It uses bagging and majority voting to improve generalization.

Implementation Details: Implemented using `RandomForestClassifier` with:

- *n_estimators* $\in \{50, 100, 200\}$
- *max_depth* $\in \{\text{None}, 20, 30\}$
- *min_samples_split*: 2 to 10
- *min_samples_leaf*: 1 to 4

Feature importances were extracted for analysis and visualization.

Observations:

- Consistently high accuracy across runs
- Lower variance than single Decision Tree
- Feature ranking aligned with biological traits (shape & texture & margin)
- Efficient parallel training across CPU cores
- Random Forest served as a reliable, interpretable, and accurate model — especially helpful for feature engineering and prototype testing.

2.5 Artificial Neural Network (ANN)

ANNs use layers of neurons to learn non-linear mappings from inputs to outputs. We built a dense feedforward network using TensorFlow/Keras.

Model Architecture:

- Input layer: 30 nodes after PCA
- Hidden layers: [256, 128] units with ReLU
- Dropout: 30% per layer to reduce overfitting
- Output layer: softmax with 99 nodes (one per class)

Training Configuration:

- Optimizer: Adam
- Loss: categorical cross-entropy
- Epochs: 50 (early stopping with patience=10)
- Batch size: 32

Observations:

- Performed competitively after fine-tuning
- Required regularization due to small dataset
- Susceptible to noise unless dimensionality was reduced
- Model was extended for online deployment via Flask (covered later)

Model Design:

- Input: Numpy arrays
- Dense $\rightarrow$ Dropout (30%) $\rightarrow$ Dense $\rightarrow$ Dropout (30%) $\rightarrow$ Output layer (softmax)

Training Strategy:

- Optimizer: Adam with learning rate 0.0001
- Early stopping on validation loss

Observations:

- Sensitive to batch size and overfitting
- More computationally expensive

2.6 Feature Reduction Techniques

To improve model performance and reduce overfitting, we applied two feature dimensionality reduction techniques.

Principal Component Analysis (PCA):

- Reduced feature space from 192 to ~ 30
- Retained $\sim 95\%$ of total variance
- Improved speed and reduced training time for ANN and SVM

Linear Discriminant Analysis (LDA):

- Supervised reduction based on class separation
- Outperformed PCA in combination with SVM
- Limited to 98 dimensions ($k - 1$ classes)

Both techniques helped models converge faster and generalize better.

2.7 Key Insights from Model Development

- **SVM** achieved the highest accuracy and F1 score among all models on structured feature data.
- **Random Forest** offered strong performance with high interpretability and stable results across different runs.
- **ANN** performed well after careful tuning, especially with dimensionality reduction and regularization to combat overfitting.
- **KNN** served as a strong baseline model but struggled with high-dimensional data and was sensitive to noise.
- **Decision Tree** helped in identifying important features but suffered from overfitting and low overall accuracy.
- **PCA and LDA** significantly improved model training speed and generalization without compromising accuracy.
- **Feature importance analysis** revealed shape-based features as the most influential, followed by texture and margin descriptors.
- Models that included dimensionality reduction generally showed better convergence and robustness.

These findings informed our final model selection and provided a foundation for integrating structured and unstructured data in future work.

3 Experiments and Results

In this section, we present a detailed account of the experiments conducted throughout the project. This includes a description of the dataset, preprocessing steps, validation strategies, model-specific configurations, evaluation metrics, and performance comparisons. The goal was to identify which techniques yield the best classification accuracy for the complex task of species identification from leaf data.

3.1 Dataset Description

The dataset used in this project was obtained from the Leaf Classification competition hosted on Kaggle. It consists of:

- 990 training samples, each representing one of 99 distinct plant species
- 594 test samples, with labels withheld for private leaderboard evaluation
- 192 engineered features per sample, capturing shape, margin, and texture characteristics

The dataset is inherently multiclass and imbalanced, with some species having fewer samples than others. This complexity necessitated robust validation techniques to ensure generalization.

3.2 Preprocessing

Effective preprocessing was critical to model performance. We tailored our preprocessing pipeline to match the needs of structured models (like SVM and Random Forest).

For Numerical Features:

- Standardization: All features were normalized using StandardScaler to ensure uniform contribution during distance or gradient-based learning.
- Dimensionality Reduction:
 - PCA (Principal Component Analysis): Reduced 192 features to ~ 30 dimensions while preserving $\sim 95\%$ variance.
 - LDA (Linear Discriminant Analysis): Used to maximize class separability before feeding into linear models like SVM.

For Image Inputs:

3.3 Validation Strategy

To rigorously assess model performance and minimize overfitting:

- Stratified 5-Fold Cross-Validation was applied to preserve class distribution across folds.
- For ANN model, a 15% validation split was created from the training data to monitor validation loss and apply early stopping.
- The Kaggle test set was reserved strictly for final performance reporting.

3.4 Evaluation Metrics

Given the multiclass nature of the problem, we evaluated each model using:

- Accuracy: Overall correct predictions as a percentage of total predictions
- F1 Score: Takes into account how the data is distributed.
- Confusion Matrix: Visualized to observe class-level prediction patterns
- Training and Inference Time: Important for practical applicability

3.5 Model Training Details

Below are training setups and results for each model:

K-Nearest Neighbors (KNN):

- Best parameters: k=5, metric=euclidean, weights=distance
- Trained on PCA-reduced data
- Validation accuracy: 96.97%
- Notable issues with scaling as dataset size grew

Support Vector Machine (SVM):

- Kernel: RBF; C=10, gamma=0.01
- Accuracy: 98.99%
- Excellent diagonal dominance in confusion matrix
- High memory usage, slower training time

Decision Tree:

- Tuned with: max_depth=20, min_samples_leaf=4
- Accuracy: 67.68%
- Highly interpretable, but prone to overfitting
- Helped in visualizing feature importance

Random Forest:

- n_estimators=100, max_depth=30
- Accuracy: 97.47%
- Robust across folds
- Extracted meaningful feature importances: Eccentricity, Solidity, Hu moments ranked highest

Artificial Neural Network (ANN):

- Architecture: Hidden Layers: [256, 128], ReLU; Dropout: 30%; Output: softmax (99 units)
- Optimizer: Adam
- Epochs: up to 50 with early stopping
- Accuracy: 98.99%
- Benefited significantly from PCA preprocessing

3.6 Comparative Model Performance

Model	Accuracy (%)	Precision (macro avg)	Recall (macro avg)	F1 Score (macro avg)
SVM	98.99	0.99	0.99	0.9892
ANN	98.99	0.99	0.99	0.9892
Random Forest	97.47	0.98	0.97	0.9731
KNN	96.97	0.97	0.97	0.9677
Decision Tree	67.68	0.68	0.68	0.6579

3.7 Confusion Matrix Insights

We analyzed the confusion matrices for SVM and ANN — our top two models:

- SVM: Misclassifications occurred mostly between morphologically similar species (e.g., Class 3 vs Class 5)
- ANN: Achieved identical accuracy to SVM with slightly different error patterns
- Both models had more than 98% diagonal values, indicating strong one-to-one mappings between predicted and true classes

3.8 Visualization of Feature Space

Using PCA, we projected high-dimensional features to 2D:

- PCA: First 3 components explained $\sim 80\%$ variance
- Visualized clusters revealed tight grouping for some species

These plots supported the notion that shape-based features hold significant discriminative power.

3.9 Learnings and Observations

Our key takeaways from the experimental phase are as follows:

- Dimensionality Reduction is Critical: PCA and LDA not only reduced training time but also improved generalization for ANN and SVM.
- Shape Features are Most Informative: Across all models, features related to eccentricity, solidity, and aspect ratio ranked highest in importance.
- SVM and ANN Tied for Top Performance: Both models achieved identical 98.99% accuracy, though with slightly different performance characteristics.
- Decision Trees Underperformed: With only 67.68% accuracy, standalone decision trees proved insufficient for this complex multiclass problem.
- Random Forests are Reliable and Interpretable: These models offered a strong balance between performance (97.47% accuracy) and explainability, making them suitable for early-stage diagnostics.

4 Learnings

The journey of building a robust and effective leaf classification system offered us a comprehensive learning experience in machine learning, data preprocessing, and performance evaluation. Our project involved a systematic exploration of a variety of algorithms on structured leaf features and yielded both impressive results and deep insights into model behavior.

4.1 Model Performance Reflection

Among all models tested, Support Vector Machine (SVM) and Artificial Neural Network (ANN) tied as the top performers, both achieving a remarkable 98.99% accuracy along with balanced precision, recall, and F1-scores of 0.9892. SVM's margin-maximizing nature and ability to handle high-dimensional feature spaces were especially beneficial in our structured dataset, while ANN's ability to model complex non-linear relationships proved equally effective after proper regularization and dimensionality reduction.

Random Forest followed closely with 97.47% accuracy and an F1-score of 0.9731, demonstrating the power of ensemble methods to improve generalization while handling non-linear boundaries and noise effectively.

K-Nearest Neighbors (KNN) performed competitively with 96.97% accuracy and 0.9677 F1-score. KNN offered simplicity and fast implementation, but required careful preprocessing to prevent issues like the curse of dimensionality and irrelevant feature influence.

Decision Tree significantly underperformed relative to the others, with only 67.68% accuracy and 0.6579 F1-score. This stark contrast underscored the importance of using ensemble techniques or regularization to improve tree-based methods when dealing with complex classification tasks.

4.2 Feature Engineering and Reduction

Feature preprocessing played a critical role across all models. Standardization ensured fair distance calculations and convergence during training. Furthermore, we implemented Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) for dimensionality reduction. PCA helped reduce redundancy in the feature space, while LDA significantly enhanced class separability for models like SVM and KNN, improving both training efficiency and performance.

4.3 Evaluation Techniques

Consistent evaluation metrics were key to objective model comparison. We used accuracy, macro-averaged precision, recall, and F1 score to capture performance across all classes, avoiding bias toward majority classes. Cross-validation and early stopping were integrated where applicable, preventing overfitting and ensuring reliability in reported metrics.

4.4 Technical Challenges and Solutions

We encountered challenges in:

- Optimizing hyperparameters across multiple models without overfitting.
- Balancing model complexity with interpretability.
- Ensuring clean and meaningful data transformation pipelines.

We addressed these through grid search, careful feature selection, and by evaluating trade-offs between performance and computational cost. Ensemble learning (via Random Forest) and regularization techniques proved highly effective in stabilizing results.

4.5 Broader Takeaways

This project sharpened our understanding of:

- How classical machine learning methods still remain powerful for structured data tasks.
- The importance of preprocessing and data scaling in distance- and margin-based algorithms.
- The value of evaluating models holistically, considering not just accuracy but precision and recall.

- Building scalable ML pipelines with room for future extensions like deep learning models and deployment.

We also gained hands-on experience with tuning, cross-validation strategies, and interpreting confusion matrices and feature importances, further enhancing our applied machine learning skillset.

4.6 Future Scope

In future work, we plan to:

- Integrate convolutional models for direct image input once the dataset is extended and cleaned further.
- Incorporate explainability tools like SHAP or LIME for black-box models.
- Move towards a deployable solution with a frontend interface once the web module is complete.

This project laid a strong foundation for our end-to-end understanding of ML workflows, from data handling to model optimization, and opened doors for deeper explorations into both classical and modern machine learning techniques.

References

- [1] Nayan Kute. *Leaf Classification using Machine Learning Algorithms*. GitHub Repository. Available at: https://github.com/Nayan-Kute21/PRML_project.git
- [2] Kaggle. *Leaf Classification Dataset*. Available at: <https://www.kaggle.com/competitions/leaf-classification/data>
- [3] Anand Mishra. *PRML Spring 2023 Project Templates and Resources*. GitHub Repository. Available at: <https://github.com/anandmishra22/PRML-Spring-2023/tree/main/Project>
- [4] Scikit-learn Developers. *Scikit-learn: Machine Learning in Python*. Available at: <https://scikit-learn.org/stable/>
- [5] Jolliffe, I.T. *Principal Component Analysis*. Springer Series in Statistics, 2nd Edition, 2002.
- [6] Cortes, C. and Vapnik, V. *Support-Vector Networks*. Machine Learning, 20(3), pp.273-297, 1995.
- [7] Breiman, L. *Random Forests*. Machine Learning, 45(1), pp.5-32, 2001.
- [8] Cover, T. and Hart, P. *Nearest Neighbor Pattern Classification*. IEEE Transactions on Information Theory, 13(1), pp.21-27, 1967.

A Contribution of each member

1. **Aditya Nerlekar (B23CS1086)**: Focused on implementing and tuning the SVM model for optimal performance. Contributed to model evaluation using confusion matrices and F1-scores. Assisted in visualizing results through PCA, and supported report writing for the Results section.
2. **Ishan (B23CS1020)**: Handled data preprocessing including standardization, PCA, and LDA. Developed the KNN model and participated in setting up stratified cross-validation. Helped analyze performance metrics and contributed to visualization of results.
3. **Nayan Kute (B23CS1041)**: Implemented the Random Forest and Decision Tree models with hyperparameter tuning. Contributed to calculating performance metrics such as accuracy, precision, and recall. Helped generate comparison plots and maintain report formatting.
4. **Neer Modi (B23CS1043)**: Conducted exploratory data analysis and dimensionality reduction using PCA and t-SNE. Assisted in building and tuning the Decision Tree and Random Forest models. Helped create visualizations and contributed to report presentation materials.

5. **Prakhar Srivastava (B23CM1056):** Participated in implementing and validating KNN and ANN. Developed and optimized the Artificial Neural Network (ANN) model. Led evaluation of ANN performance using cross-validation and key metrics. Integrated model predictions into a Flask web application and supported final presentation development.